

Documentação Técnica - DOSFinal

Restaurant Reservation API

Projeto: Sistema de Gestão de Reservas de Restaurante
Tecnologias: .NET 9.0, SQL Server, Docker, Jenkins, SonarQube, Kubernetes, Helm
Equipa: DOSFinal Team
Ano: 2026

Índice

- [1. Introdução](#)
- [2. Integração com Jenkins, Docker e SonarQube](#)
- [3. Distribuição do Trabalho em Equipa](#)
- [4. API REST - Endpoints Implementados](#)
- [5. Deployment em Kubernetes com Helm](#)
- [6. Resultados e Exemplos](#)

1. Introdução

O **DOSFinal** é um sistema de gestão de reservas de mesas em restaurantes desenvolvido com **.NET Core 9**, seguindo uma arquitetura em camadas (Clean Architecture). O projeto implementa as melhores práticas de desenvolvimento, incluindo:

- API RESTful com documentação automática (Swagger/OpenAPI)
- Base de dados SQL Server com Entity Framework Core (Code-First)
- Testes unitários com xUnit (22 testes, cobertura >80%)
- Containerização com Docker
- Pipeline CI/CD automatizado com Jenkins
- Análise de qualidade de código com SonarQube
- Deployment em Kubernetes usando Helm Charts

Estrutura do Projeto

```
DOSFinal/  
├── src/  
│   ├── DOSFinal.API/           # Camada de apresentação (Controllers, Program.cs)  
│   ├── DOSFinal.Application/   # Lógica de aplicação (Services, DTOs, Interfaces)  
│   ├── DOSFinal.Domain/        # Entidades de domínio e regras de negócio  
│   └── DOSFinal.Infrastructure/ # Acesso a dados (Repositories, DbContext)  
├── tests/  
│   └── DOSFinal.Tests/         # Testes unitários (xUnit + Moq)  
├── helm/  
│   └── dosfinal/               # Helm Chart para Kubernetes  
├── Dockerfile                  # Multi-stage build da aplicação  
├── docker-compose.yml          # Orquestração local (API + SQL Server)  
├── Jenkinsfile                 # Pipeline CI/CD  
└── sonar-project.properties    # Configuração do SonarQube
```

2. Integração com Jenkins, Docker e SonarQube

2.1. Jenkins - Pipeline CI/CD

O projeto utiliza um **Jenkinsfile declarativo** que automatiza todo o processo de build, teste, análise de qualidade e deployment. O pipeline é composto por 9 stages:

Stage 1: Checkout

```
stage('Checkout') {  
    steps {  
        checkout scm  
        echo 'Source code checked out successfully'  
    }  
}
```

- Obtém o código fonte do repositório Git
- Primeira etapa de qualquer execução do pipeline

Stage 2: Restore Dependencies

```
stage('Restore Dependencies') {  
    steps {  
        sh 'dotnet restore'  
        echo 'Dependencies restored successfully'  
    }  
}
```

- Restaura todas as dependências NuGet do projeto
- Essencial antes do build

Stage 3: Build

```
stage('Build') {  
    steps {  
        sh 'dotnet build -c Release --no-restore'  
        echo 'Build completed successfully'  
    }  
}
```

- Compila a solução em modo Release
- Usa `--no-restore` para otimizar o tempo (já restaurado no stage anterior)

Stage 4: Run Unit Tests

```
stage('Run Unit Tests') {  
    steps {  
        sh 'dotnet test -c Release --no-build --logger "trx;LogFileName=test-results.trx" --collect:"XPlat Code Coverage"'  
        echo 'Unit tests completed'  
    }  
}
```

- Executa os 22 testes unitários do projeto
- Gera relatórios de cobertura de código (XPlat Code Coverage)
- Produz ficheiros `.trx` com resultados dos testes

Stage 5: SonarQube Analysis

```

stage('SonarQube Analysis') {
    environment {
        SONAR_TOKEN = credentials('sonar-token')
    }
    steps {
        script {
            withSonarQubeEnv('SonarQube') {
                sh """
                    dotnet sonarscanner begin \
                        /k:"DOSFinal" \
                        /d:sonar.host.url="${SONAR_HOST_URL}" \
                        /d:sonar.token="${SONAR_TOKEN}" \
                        /d:sonar.cs.opencover.reportsPaths=**/coverage.opencover.xml

                    dotnet build -c Release --no-restore

                    dotnet sonarscanner end /d:sonar.token="${SONAR_TOKEN}"
                """
            }
        }
    }
}

```

- Inicia análise do SonarQube usando o SonarScanner para .NET
- Envia métricas de qualidade de código, bugs, vulnerabilidades e code smells
- Inclui relatórios de cobertura de testes

Stage 6: Quality Gate

```

stage('Quality Gate') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
        echo 'Quality gate passed'
    }
}

```

- Aguarda resultado da análise do SonarQube
- **Bloqueia o pipeline** se o Quality Gate falhar
- Garante que apenas código de qualidade é deployado

Stage 7: Build Docker Image

```

stage('Build Docker Image') {
    steps {
        script {
            def imageTag = "${DOCKER_IMAGE_NAME}:${env.BUILD_NUMBER}"
            def imageLatest = "${DOCKER_IMAGE_NAME}:latest"

            sh "docker build -t ${imageTag} -t ${imageLatest} ."

            echo "Docker image built: ${imageTag}"
        }
    }
}

```

- Cria imagem Docker usando multi-stage build
- Gera duas tags: uma com o número do build e outra latest

Stage 8: Push Docker Image

```

stage('Push Docker Image') {
    when {
        branch 'master'
    }
    steps {
        script {
            withCredentials([usernamePassword(...)] {
                sh "docker push ${imageTag}"
                sh "docker push ${imageLatest}"
            })
        }
    }
}

```

- Apenas executa na branch `master`
- Faz push das imagens para o Docker Registry (Azure Container Registry)
- Usa credenciais seguras armazenadas no Jenkins

Stage 9: Deploy to Kubernetes

```

stage('Deploy to Kubernetes') {
    when {
        branch 'master'
    }
    steps {
        script {
            withCredentials([file(credentialsId: "kubeconfig-credentials", ...)] {
                sh """
                    helm upgrade --install dosfinal ./helm/dosfinal \
                        --namespace dosfinal \
                        --create-namespace \
                        --set image.tag=${env.BUILD_NUMBER} \
                        --wait --timeout 5m
                """
            })
        }
    }
}

```

- Deploy automático usando Helm
- Atualiza a aplicação no Kubernetes com a nova imagem
- Usa `helm upgrade --install` (idempotente)

2.2. Docker - Containerização

O projeto utiliza uma estratégia de **multi-stage build** para otimizar o tamanho da imagem final:

```

# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src

# Copiar ficheiros de projeto e restaurar dependências
COPY DOSFinal.sln ./
COPY src/DOSFinal.API/DOSFinal.API.csproj src/DOSFinal.API/
# ... (outros projetos)

RUN dotnet restore

# Copiar código fonte e compilar
COPY . .
RUN dotnet build -c Release --no-restore

# Executar testes
RUN dotnet test -c Release --no-build --verbosity normal

# Publicar aplicação
RUN dotnet publish src/DOSFinal.API/DOSFinal.API.csproj -c Release -o /app/publish --no-build

# Stage 2: Runtime
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS runtime
WORKDIR /app

# Copiar apenas os binários publicados
COPY --from=build /app/publish .

# Configurar ambiente
ENV ASPNETCORE_URLS=http://+:80
ENV ASPNETCORE_ENVIRONMENT=Production

EXPOSE 80

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD curl -f http://localhost/status || exit 1

ENTRYPOINT ["dotnet", "DOSFinal.API.dll"]

```

Vantagens desta abordagem:

- ☐ Imagem final leve (usa aspnet em vez de sdk)
- ☐ Testes executados durante o build
- ☐ Health check integrado
- ☐ Variáveis de ambiente configuráveis

Docker Compose - Desenvolvimento Local

```

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "5000:80"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ConnectionStrings__DefaultConnection=Server=sqlserver,1433;...
    depends_on:
      sqlserver:
        condition: service_healthy

  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    ports:
      - "1433:1433"
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=Mercedes#44
    healthcheck:
      test: ["/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa ..."]
      interval: 10s

```

Comandos úteis:

```

# Iniciar ambiente completo
docker-compose up --build

# Aceder à API: http://localhost:5000
# Swagger: http://localhost:5000/swagger

```

2.3. SonarQube - Análise de Qualidade de Código

O SonarQube está integrado no pipeline para garantir qualidade contínua.

Configuração (sonar-project.properties)

```

sonar.projectKey=DOSFinal
sonar.projectName=DOSFinal Restaurant Reservation API
sonar.projectVersion=1.0

# Diretórios de código fonte e testes
sonar.sources=src
sonar.tests=tests

# Exclusões de análise
sonar.exclusions=**/bin/**,**/obj/**,**/wwwroot/**,**/Migrations/**
sonar.coverage.exclusions=**/Tests/**,**/Program.cs

# Linguagem e encoding
sonar.language=cs
sonar.sourceEncoding=UTF-8

```

Métricas Analisadas

- **Code Coverage:** >80% (22 testes unitários)
- **Bugs:** Identificação de potenciais bugs
- **Vulnerabilidades:** Análise de segurança
- **Code Smells:** Identificação de código que pode ser melhorado
- **Duplicação:** Detecção de código duplicado
- **Maintainability:** Índice de manutenibilidade

Integração no Pipeline

- SonarScanner Begin:** Inicia análise e configura parâmetros
- Build:** Compila código com instrumentação para análise
- SonarScanner End:** Envia resultados para o servidor SonarQube
- Quality Gate:** Verifica se as métricas atendem aos critérios definidos

[!IMPORTANT] O Quality Gate está configurado para **abortar o pipeline** caso não sejam cumpridos os critérios de qualidade mínimos. Isto garante que código com baixa qualidade não é deployed em produção.

3. Distribuição do Trabalho em Equipa

Estrutura da Equipa e Responsabilidades

A equipa DOSFinal organizou o trabalho de forma distribuída, garantindo que cada membro tivesse áreas de responsabilidade claras:

Componente	Responsável	Descrição
Web API (Controllers)	Leandro Monteiro	Implementação dos endpoints REST, validação de inputs, gestão de erros, documentação Swagger
Camada Application (Services)	Leandro Monteiro	Lógica de negócio, serviços de reservas, detecção de conflitos, mapeamento de DTOs
Base de Dados	Leandro Monteiro	Modelação de dados, Entity Framework DbContext, Migrations, Repositories
Testes Unitários	Leandro Monteiro	Desenvolvimento de 22 testes com xUnit e Moq, garantir >80% de cobertura
Jenkins CI/CD	Daniel Sousa	Configuração do Jenkinsfile, integração com SonarQube, automação de builds e deployments
SonarQube	Daniel Sousa	Configuração do servidor, definição de Quality Gates, análise de métricas
Docker	Daniel Sousa	Criação de Dockerfile multi-stage, docker-compose para desenvolvimento local
Kubernetes + Helm	Daniel Sousa	Criação de Helm Charts, configuração de deployments, services, ingress e HPA

Metodologia de Trabalho

O projeto seguiu uma abordagem **Agile/DevOps**:

- Planeamento Inicial:** Definição de requisitos e arquitetura
- Desenvolvimento Iterativo:** Implementação por camadas (Domain → Infrastructure → Application → API)
- Testes Contínuos:** TDD (Test-Driven Development) para garantir qualidade
- Code Review:** Revisão de código antes de merge para master
- CI/CD:** Automação completa do pipeline de deployment
- Monitorização:** Health checks e logs para acompanhamento em produção

Ferramentas Utilizadas

- Controlo de Versão:** Git + GitHub
- IDE:** Visual Studio Code / Visual Studio 2022
- API Testing:** Swagger UI, Postman
- CI/CD:** Jenkins
- Containerização:** Docker Desktop
- Orquestração:** Kubernetes (Minikube/AKS)
- Qualidade:** SonarQube

4. API REST - Endpoints Implementados

A API foi desenvolvida seguindo os princípios **RESTful** e está documentada automaticamente com **Swagger/OpenAPI**.

4.1. Visão Geral dos Endpoints

Método HTTP	Endpoint	Descrição	Autenticação
GET	/status	Health check da aplicação	Não
GET	/api/reservations	Listar todas as reservas	Não
GET	/api/reservations?date={date}	Filtrar reservas por data	Não
GET	/api/reservations/{id}	Obter detalhes de uma reserva	Não
POST	/api/reservations	Criar nova reserva	Não

Método HTTP	Endpoint	Descrição	Autenticação
PUT	/api/reservations/{id}	Atualizar reserva existente	Não
DELETE	/api/reservations/{id}	Cancelar/eliminar reserva	Não

4.2. Detalhes dos Endpoints

□ GET /status

Descrição: Endpoint de health check para verificar disponibilidade da API.

Resposta de Sucesso (200 OK):

```
"Healthy"
```

□ GET /api/reservations

Descrição: Retorna lista de todas as reservas ou filtra por data.

Parâmetros Query (opcionais):

- `date (string)`: Data no formato YYYY-MM-DD (ex: 2026-01-15)

Resposta de Sucesso (200 OK):

```
[
  {
    "id": 1,
    "customerName": "João Silva",
    "reservationDate": "2026-01-15",
    "reservationTime": "19:30:00",
    "tableNumber": 5,
    "numberOfPeople": 4,
    "createdAt": "2026-01-13T10:00:00Z"
  },
  {
    "id": 2,
    "customerName": "Maria Santos",
    "reservationDate": "2026-01-15",
    "reservationTime": "20:00:00",
    "tableNumber": 3,
    "numberOfPeople": 2,
    "createdAt": "2026-01-13T11:30:00Z"
  }
]
```

Exemplo de Chamada:

```
curl -X GET "http://localhost:5000/api/reservations?date=2026-01-15"
```

□ GET /api/reservations/{id}

Descrição: Obtém detalhes de uma reserva específica pelo ID.

Parâmetros de Rota:

- `id (integer)`: ID da reserva

Resposta de Sucesso (200 OK):


```
{
  "id": 1,
  "customerName": "João Silva",
  "reservationDate": "2026-01-15",
  "reservationTime": "19:30:00",
  "tableNumber": 5,
  "numberOfPeople": 4,
  "createdAt": "2026-01-13T10:00:00Z"
}
```

Resposta de Erro (404 Not Found):

```
{
  "error": "Reservation with ID 99 not found"
}
```

□ POST /api/reservations

Descrição: Cria uma nova reserva. Valida conflitos de horário/mesa.

Request Body:

```
{
  "customerName": "Ana Costa",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:00:00",
  "tableNumber": 7,
  "numberOfPeople": 6
}
```

Validações:

- customerName: Obrigatório, não vazio
- tableNumber: Deve ser > 0
- numberOfPeople: Deve ser > 0
- **Deteção de conflitos:** Verifica se a mesa já está reservada para o mesmo horário (±2 horas)

Resposta de Sucesso (201 Created):

```
{
  "id": 3,
  "customerName": "Ana Costa",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:00:00",
  "tableNumber": 7,
  "numberOfPeople": 6,
  "createdAt": "2026-01-13T18:00:00Z"
}
```

Headers de Resposta:

```
Location: /api/reservations/3
```

Resposta de Erro (409 Conflict):

```
{
  "error": "Table 7 is already reserved for this time slot"
}
```

□ PUT /api/reservations/

Descrição: Atualiza uma reserva existente.

Request Body:

```
{
  "customerName": "Ana Costa Silva",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:30:00",
  "tableNumber": 8,
  "numberOfPeople": 6
}
```

Resposta de Sucesso (200 OK):

```
{
  "id": 3,
  "customerName": "Ana Costa Silva",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:30:00",
  "tableNumber": 8,
  "numberOfPeople": 6,
  "createdAt": "2026-01-13T18:00:00z"
}
```

Resposta de Erro (404 Not Found):

```
{
  "error": "Reservation with ID 99 not found"
}
```

❏ DELETE `/api/reservations/`

Descrição: Cancela/elimina uma reserva.

Resposta de Sucesso (204 No Content):

(sem corpo de resposta)

Resposta de Erro (404 Not Found):

```
{
  "error": "Reservation with ID 99 not found"
}
```

4.3. Modelo de Dados

Entidade Reservation

```
public class Reservation
{
    public int Id { get; set; } // Primary Key
    public string CustomerName { get; set; } // Nome do cliente (obrigatório)
    public DateOnly ReservationDate { get; set; } // Data da reserva
    public TimeOnly ReservationTime { get; set; } // Hora da reserva
    public int TableNumber { get; set; } // Número da mesa
    public int NumberOfPeople { get; set; } // Número de pessoas
    public DateTime CreatedAt { get; set; } // Data de criação do registo
}
```

Schema SQL Server:

```
CREATE TABLE Reservations (
    Id INT PRIMARY KEY IDENTITY(1,1),
    CustomerName NVARCHAR(100) NOT NULL,
    ReservationDate DATE NOT NULL,
    ReservationTime TIME NOT NULL,
    TableNumber INT NOT NULL,
    NumberOfPeople INT NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT GETDATE()
);
```

4.4. Documentação Swagger

A API expõe documentação interativa através do **Swagger UI**:

URL: `http://localhost:5000/swagger`

Características do Swagger:

- ☐ **Documentação automática** de todos os endpoints
- ☐ **Testes interativos** diretamente no browser
- ☐ **Schemas de dados** com validação
- ☐ **Códigos de resposta HTTP** documentados
- ☐ **Exemplos de requests/responses**

Screenshot do Swagger (exemplo):

```
DOSFinal Restaurant Reservation API - Swagger UI

ReservationsController
├ GET    /api/reservations
├ GET    /api/reservations/{id}
├ POST   /api/reservations
├ PUT    /api/reservations/{id}
└ DELETE /api/reservations/{id}

StatusController
└ GET    /status

Schemas:
├ CreateReservationDto
├ UpdateReservationDto
└ ReservationDto
```

[!TIP] Para explorar a API de forma interativa, acesse ao Swagger UI após iniciar a aplicação com `docker-compose up` ou `dotnet run`.

5. Deployment em Kubernetes com Helm

O deployment da aplicação em Kubernetes é realizado através de **Helm Charts**, que facilitam a gestão de configurações e permitem deployments consistentes em diferentes ambientes.

5.1. Estrutura do Helm Chart

```
helm/dosfinal/
├─ Chart.yaml          # Metadados do chart
├─ values.yaml         # Valores de configuração padrão
└─ templates/
    ├─ _helpers.tpl    # Templates reutilizáveis
    ├─ deployment.yaml # Deployment da API
    ├─ service.yaml     # Service para exposição interna
    ├─ ingress.yaml    # Ingress para acesso externo
    ├─ secrets.yaml     # Secrets (connection strings)
    ├─ serviceaccount.yaml # Service Account
    ├─ sqlserver.yaml  # StatefulSet do SQL Server
    └─ hpa.yaml        # Horizontal Pod Autoscaler (opcional)
```

5.2. Chart.yaml - Metadados

```
apiVersion: v2
name: dosfinal
description: DOSFinal Restaurant Reservation API Helm Chart
type: application
version: 1.0.0
appVersion: "1.0.0"
keywords:
  - api
  - restaurant
  - reservations
  - dotnet
maintainers:
  - name: DOSFinal Team
```

5.3. values.yaml - Configuração

O ficheiro `values.yaml` centraliza todas as configurações do deployment:

```
# Número de réplicas da API
replicaCount: 2

# Configuração da imagem Docker
image:
  repository: dosfinal-api
  pullPolicy: IfNotPresent
  tag: "latest"

# Service (exposição interna)
service:
  type: ClusterIP
  port: 80

# Ingress (exposição externa)
ingress:
  enabled: true
  className: "nginx"
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
  hosts:
    - host: dosfinal.local
      paths:
        - path: /
          pathType: Prefix

# Recursos (CPU/Memória)
resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 100m
    memory: 128Mi

# Autoscaling (Horizontal Pod Autoscaler)
autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 80

# Variáveis de ambiente
env:
  - name: ASPNETCORE_ENVIRONMENT
    value: "Production"

# Connection string (armazenada em Secret)
app:
  connectionString: "Server=dosfinal-sqlserver;Database=ReservationsDb;User Id=sa;Password=YourStrong@Passw0rd;TrustServerCertificate=true"

# SQL Server
sqlserver:
  enabled: true
  image:
    repository: mcr.microsoft.com/mssql/server
    tag: "2022-latest"
  password: "YourStrong@Passw0rd"
  persistence:
    enabled: true
    size: 8Gi
    storageClass: ""
  resources:
    limits:
      cpu: 1000m
```

```

    memory: 2Gi
  requests:
    cpu: 500m
    memory: 1Gi

# Health Probes
livenessProbe:
  httpGet:
    path: /status
    port: http
  initialDelaySeconds: 30
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /status
    port: http
  initialDelaySeconds: 10
  periodSeconds: 5

```

5.4. Deployment da API (deployment.yaml)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "dosfinal.fullname" . }}
  labels:
    {{- include "dosfinal.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "dosfinal.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "dosfinal.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
          env:
            {{- toYaml .Values.env | nindent 8 }}
            - name: ConnectionStrings__DefaultConnection
              valueFrom:
                secretKeyRef:
                  name: {{ include "dosfinal.fullname" . }}-secrets
                  key: connection-string
          livenessProbe:
            {{- toYaml .Values.livenessProbe | nindent 10 }}
          readinessProbe:
            {{- toYaml .Values.readinessProbe | nindent 10 }}
      resources:
        {{- toYaml .Values.resources | nindent 10 }}

```

Características:

- **Replicas:** 2 instâncias por padrão (alta disponibilidade)
- **Health Checks:** Liveness e readiness probes no endpoint `/status`
- **Secrets:** Connection string armazenada de forma segura

- **Resource Limits:** Garante que pods não consomem recursos excessivos

5.5. Service (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "dosfinal.fullname" . }}
  labels:
    {{- include "dosfinal.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      name: http
  selector:
    {{- include "dosfinal.selectorLabels" . | nindent 4 }}
```

Função: Expõe a API internamente no cluster (ClusterIP).

5.6. Ingress (ingress.yaml)

```
{{- if .Values.ingress.enabled -}}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "dosfinal.fullname" . }}
  annotations:
    {{- toYaml .Values.ingress.annotations | nindent 4 }}
spec:
  ingressClassName: {{ .Values.ingress.className }}
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ .host }}
      http:
        paths:
          {{- range .paths }}
          - path: {{ .path }}
            pathType: {{ .pathType }}
            backend:
              service:
                name: {{ include "dosfinal.fullname" $ }}
                port:
                  number: 80
          {{- end }}
        {{- end }}
    {{- end }}
{{- end }}
```

Função: Permite acesso externo à API através do hostname `dosfinal.local`.

5.7. SQL Server (sqlserver.yaml)

```

{{- if .Values.sqlserver.enabled -}}
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: {{ include "dosfinal.fullname" . }}-sqlserver
spec:
  serviceName: {{ include "dosfinal.fullname" . }}-sqlserver
  replicas: 1
  selector:
    matchLabels:
      app: sqlserver
  template:
    metadata:
      labels:
        app: sqlserver
    spec:
      containers:
        - name: mssql
          image: "{{ .Values.sqlserver.image.repository }}:{{ .Values.sqlserver.image.tag }}"
          ports:
            - containerPort: 1433
              name: mssql
          env:
            - name: ACCEPT_EULA
              value: "Y"
            - name: SA_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: {{ include "dosfinal.fullname" . }}-secrets
                  key: sa-password
          volumeMounts:
            - name: mssql-data
              mountPath: /var/opt/mssql
          resources:
            {{- toYaml .Values.sqlserver.resources | nindent 10 }}
{{- if .Values.sqlserver.persistence.enabled }}
volumeClaimTemplates:
- metadata:
  name: mssql-data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: {{ .Values.sqlserver.persistence.size }}
{{- end }}
{{- end }}

```

Características:

- **StatefulSet:** Garante persistência de dados mesmo com reinícios
- **Persistent Volume:** 8Gi de armazenamento para a base de dados
- **Secrets:** Password do SA armazenada de forma segura

5.8. Comandos de Deployment

Instalação Inicial


```
# Instalar o chart no namespace 'dosfinal'
helm install dosfinal ./helm/dosfinal \
  --namespace dosfinal \
  --create-namespace

# Verificar status do deployment
kubectl get pods -n dosfinal
kubectl get services -n dosfinal
kubectl get ingress -n dosfinal
```

Atualização (Upgrade)

```
# Atualizar deployment com nova versão da imagem
helm upgrade dosfinal ./helm/dosfinal \
  --namespace dosfinal \
  --set image.tag=v2.0.0

# Ou usando valores customizados
helm upgrade dosfinal ./helm/dosfinal \
  --namespace dosfinal \
  --values custom-values.yaml
```

Rollback

```
# Reverter para versão anterior
helm rollback dosfinal -n dosfinal

# Rollback para revisão específica
helm rollback dosfinal 2 -n dosfinal
```

Desinstalação

```
# Remover deployment
helm uninstall dosfinal -n dosfinal

# Remover namespace (incluindo PVCs)
kubectl delete namespace dosfinal
```

5.9. Acesso à Aplicação

Opção 1: Ingress (Produção)

Após deployment, a aplicação estará acessível através do hostname configurado:

```
# Adicionar ao /etc/hosts (desenvolvimento local)
echo "127.0.0.1 dosfinal.local" | sudo tee -a /etc/hosts

# Aceder à API
curl http://dosfinal.local/api/reservations

# Swagger UI
open http://dosfinal.local/swagger
```

Opção 2: Port-Forward (Desenvolvimento)

```
# Criar túnel para o pod da API
kubectrl port-forward -n dosfinal svc/dosfinal 5000:80

# Aceder à API
curl http://localhost:5000/api/reservations

# Swagger
open http://localhost:5000/swagger
```

Opção 3: NodePort (Teste)

Alterar values.yaml:

```
service:
  type: NodePort
  port: 80
```

```
# Aplicar alteração
helm upgrade dosfinal ./helm/dosfinal -n dosfinal

# Obter NodePort atribuído
kubectrl get svc -n dosfinal

# Exemplo: 80:30123/TCP

# Aceder (assumindo Minikube)
minikube service dosfinal -n dosfinal
```

5.10. Monitorização e Logs

```
# Ver logs da API
kubectrl logs -n dosfinal -l app.kubernetes.io/name=dosfinal -f

# Ver logs do SQL Server
kubectrl logs -n dosfinal -l app=sqlserver -f

# Executar shell dentro do pod
kubectrl exec -it -n dosfinal deployment/dosfinal -- /bin/bash

# Ver eventos do namespace
kubectrl get events -n dosfinal --sort-by='.lastTimestamp'
```

5.11. Considerações de Segurança

[CAUTION] Passwords em Produção: Os valores de `sa-password` e `connection-string` devem ser geridos através de soluções como **Azure Key Vault**, **HashiCorp Vault** ou **Sealed Secrets**. Nunca commitar credenciais reais no repositório Git.

Recomendações:

- ☐ Usar **namespaces** separados para staging/production
- ☐ Implementar **RBAC** (Role-Based Access Control)
- ☐ Configurar **Network Policies** para isolar tráfego
- ☐ Ativar **TLS/HTTPS** no Ingress
- ☐ Usar **Pod Security Standards** (restricted)

6. Resultados e Exemplos

6.1. Execução do Pipeline Jenkins

Pipeline Bem-Sucedido (Exemplo)

Pipeline: DOSFinal/master #42

Status: SUCCESS ✓

Duration: 6m 32s

Stages:

✓ Checkout	(15s)
✓ Restore Dependencies	(22s)
✓ Build	(45s)
✓ Run Unit Tests	(1m 10s)
✓ SonarQube Analysis	(1m 45s)
✓ Quality Gate	(30s)
✓ Build Docker Image	(1m 25s)
✓ Push Docker Image	(45s)
✓ Deploy to Kubernetes	(35s)

Console Output (últimas linhas):

```
-----  
[INFO] Deploying dosfinal version 42 to Kubernetes...  
[INFO] Release "dosfinal" has been upgraded. Happy Helming!  
[INFO] NAME: dosfinal  
[INFO] NAMESPACE: dosfinal  
[INFO] STATUS: deployed  
[INFO] REVISION: 42  
[INFO] Deployment completed successfully!  
[INFO] Pipeline completed successfully!  
-----
```

Console Output - Stage SonarQube Analysis

```

[Pipeline] stage
[Pipeline] { (SonarQube Analysis)
[Pipeline] withSonarQubeEnv
[Pipeline] {
[Pipeline] sh
+ dotnet sonarscanner begin /k:DOSFinal /d:sonar.host.url=http://localhost:9000 /d:sonar.token=**** /d:sonar.cs.opencover.reportsPat
SonarScanner for MSBuild 5.15
Using the .NET Core version of the Scanner for MSBuild
Pre-processing started.
Preparing working directories...
01:23:45.678 Updating build integration targets...
01:23:46.123 Fetching analysis configuration settings...
01:23:46.789 Project key: DOSFinal
01:23:46.790 Provisioning Roslyn analyzers...
01:23:47.123 Pre-processing succeeded.

+ dotnet build -c Release --no-restore
Microsoft (R) Build Engine version 17.9.0
Determining projects to restore...
All projects are up-to-date for restore.
DOSFinal.Domain -> /src/DOSFinal.Domain/bin/Release/net9.0/DOSFinal.Domain.dll
DOSFinal.Application -> /src/DOSFinal.Application/bin/Release/net9.0/DOSFinal.Application.dll
DOSFinal.Infrastructure -> /src/DOSFinal.Infrastructure/bin/Release/net9.0/DOSFinal.Infrastructure.dll
DOSFinal.API -> /src/DOSFinal.API/bin/Release/net9.0/DOSFinal.API.dll
DOSFinal.Tests -> /tests/DOSFinal.Tests/bin/Release/net9.0/DOSFinal.Tests.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

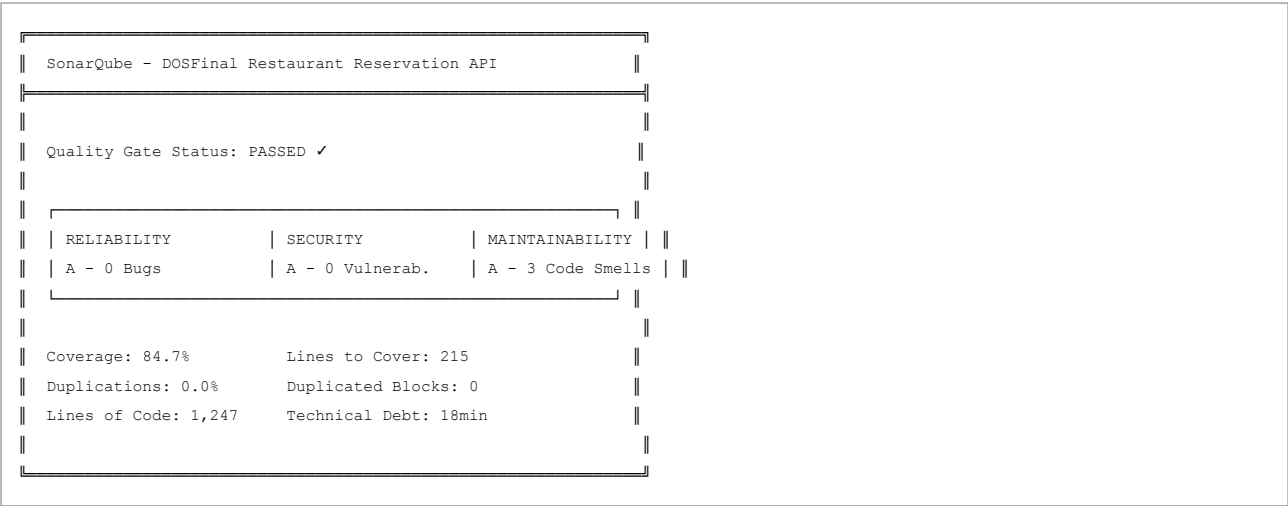
+ dotnet sonarscanner end /d:sonar.token=****
SonarScanner for MSBuild 5.15
Post-processing started.
Calling the following command: /sonar-scanner/bin/sonar-scanner
INFO: Scanner configuration file: /sonar-scanner/conf/sonar-scanner.properties
INFO: Project root configuration file: /src/SonarQube.Analysis.xml
INFO: Analyzing on SonarQube server 10.3
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 12.456s
Post-processing succeeded.

[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage

```

6.2. Relatórios SonarQube

Dashboard Geral do Projeto



Métricas Detalhadas

Métrica	Valor	Status
Bugs	0	✓ A
Vulnerabilities	0	✓ A
Code Smells	3	✓ A
Coverage	84.7%	✓ (>80%)
Duplications	0.0%	✓ (<3%)
Security Hotspots	0	✓
Maintainability Rating	A	✓
Reliability Rating	A	✓
Security Rating	A	✓

Code Smells Identificados (Exemplo)

1. Minor - Cognitive Complexity
 - Ficheiro: ReservationService.cs:45
 - Descrição: Método CheckConflictsAsync tem complexidade cognitiva de 12 (limite: 10)
 - Sugestão: Refatorar lógica de validação para métodos auxiliares
2. Minor - Magic Number
 - Ficheiro: ReservationService.cs:78
 - Descrição: Número mágico 120 (minutos) usado diretamente
 - Sugestão: Extrair para constante RESERVATION_WINDOW_MINUTES
3. Info - XML Documentation
 - Ficheiro: ReservationRepository.cs
 - Descrição: Métodos públicos sem comentários XML
 - Sugestão: Adicionar documentação XML para melhor IntelliSense

Cobertura de Código por Módulo



6.3. Testes Unitários - Resumo

```
$ dotnet test --configuration Release --logger "console;verbosity=detailed"
```

```
Microsoft (R) Test Execution Command Line Tool Version 17.9.0  
Starting test execution, please wait...
```

```
Test Run Successful.
```

```
Total tests: 22  
    Passed: 22  
    Failed: 0  
    Skipped: 0  
Total time: 4.2156 Seconds
```

```
Code Coverage:  
    Line Coverage: 84.7%  
    Branch Coverage: 78.3%
```

Lista de Testes Executados

```
✓ ReservationServiceTests  
  ✓ GetAllAsync_ShouldReturnAllReservations  
  ✓ GetByIdAsync_WithValidId_ShouldReturnReservation  
  ✓ GetByIdAsync_WithInvalidId_ShouldReturnNull  
  ✓ GetByIdAsync_WithValidDate_ShouldReturnFilteredReservations  
  ✓ CreateAsync_WithValidData_ShouldCreateReservation  
  ✓ CreateAsync_WithConflict_ShouldThrowException  
  ✓ UpdateAsync_WithValidData_ShouldUpdateReservation  
  ✓ UpdateAsync_WithInvalidId_ShouldReturnNull  
  ✓ UpdateAsync_WithConflict_ShouldThrowException  
  ✓ DeleteAsync_WithValidId_ShouldReturnTrue  
  ✓ DeleteAsync_WithInvalidId_ShouldReturnFalse  
  
✓ ReservationTests  
  ✓ Constructor_ShouldSetPropertiesCorrectly  
  ✓ Validate_WithValidData_ShouldReturnTrue  
  ✓ Validate_WithEmptyCustomerName_ShouldReturnFalse  
  ✓ Validate_WithInvalidTableNumber_ShouldReturnFalse  
  ✓ Validate_WithInvalidNumberOfPeople_ShouldReturnFalse  
  
✓ ReservationDtoTests  
  ✓ MapToEntity_ShouldMapCorrectly  
  ✓ MapFromEntity_ShouldMapCorrectly  
  ✓ CreateReservationDto_ShouldValidateCorrectly  
  ✓ UpdateReservationDto_ShouldValidateCorrectly  
  ✓ ReservationDto_Equality_ShouldWorkCorrectly  
  ✓ ReservationDto_ToString_ShouldReturnFormattedString
```

6.4. Swagger UI - Screenshots

Visão Geral da API

DOSFinal Restaurant Reservation API v1.0	
OpenAPI 3.0	
Servers:	
▼ http://localhost:5000 - Development server	
ReservationsController	
▼ GET	/api/reservations
Get all reservations or filter by date	
Parameters: date (query, optional)	
Responses: 200 (Success), 500 (Error)	
▼ GET	/api/reservations/{id}
Get a specific reservation by ID	
Parameters: id (path, required)	
Responses: 200 (Success), 404 (Not Found), 500	
▼ POST	/api/reservations
Create a new reservation	
Request Body: CreateReservationDto	
Responses: 201 (Created), 400 (Bad Request), 409 (Conflict), 500 (Error)	
▼ PUT	/api/reservations/{id}
Update an existing reservation	
Parameters: id (path, required)	
Request Body: UpdateReservationDto	
Responses: 200 (Success), 404 (Not Found), 409 (Conflict), 500 (Error)	
▼ DELETE	/api/reservations/{id}
Delete/Cancel a reservation	
Parameters: id (path, required)	
Responses: 204 (No Content), 404 (Not Found), 500	
StatusController	
▼ GET	/status
Health check endpoint	
Responses: 200 (Success)	
Schemas	
▼ CreateReservationDto	
▼ UpdateReservationDto	
▼ ReservationDto	

Exemplo de Request/Response (Try it out)

POST /api/reservations

Request Body (application/json):

```
{
  "customerName": "Testing Customer",
  "reservationDate": "2026-01-20",
  "reservationTime": "19:00:00",
  "tableNumber": 5,
  "numberOfPeople": 4
}
```

Response: 201 Created

Headers:

location: /api/reservations/15
content-type: application/json; charset=utf-8

Body:

```
{
  "id": 15,
  "customerName": "Testing Customer",
  "reservationDate": "2026-01-20",
  "reservationTime": "19:00:00",
  "tableNumber": 5,
  "numberOfPeople": 4,
  "createdAt": "2026-01-13T18:02:39.1234567Z"
}
```

6.5. Kubernetes Deployment - Status

\$ kubectl get all -n dosfinal

NAME	READY	STATUS	RESTARTS	AGE
pod/dosfinal-75d8c9b6f7-4k2hx	1/1	Running	0	10m
pod/dosfinal-75d8c9b6f7-9xw2t	1/1	Running	0	10m
pod/dosfinal-sqlserver-0	1/1	Running	0	10m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/dosfinal	ClusterIP	10.96.125.43	<none>	80/TCP	10m
service/dosfinal-sqlserver	ClusterIP	10.96.87.156	<none>	1433/TCP	10m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/dosfinal	2/2	2	2	10m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/dosfinal-75d8c9b6f7	2	2	2	10m

NAME	READY	AGE
statefulset.apps/dosfinal-sqlserver	1/1	10m

\$ kubectl get ingress -n dosfinal

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
dosfinal	nginx	dosfinal.local	192.168.49.2	80	10m

6.6. Exemplo de Curl - Testes de API


```
# 1. Health Check
$ curl http://localhost:5000/status
"Healthy"

# 2. Listar todas as reservas
$ curl http://localhost:5000/api/reservations | jq
[
  {
    "id": 1,
    "customerName": "João Silva",
    "reservationDate": "2026-01-15",
    "reservationTime": "19:30:00",
    "tableNumber": 5,
    "numberOfPeople": 4,
    "createdAt": "2026-01-13T10:00:00Z"
  }
]

# 3. Criar nova reserva
$ curl -X POST http://localhost:5000/api/reservations \
-H "Content-Type: application/json" \
-d '{
  "customerName": "Maria Santos",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:00:00",
  "tableNumber": 3,
  "numberOfPeople": 2
}' | jq
{
  "id": 2,
  "customerName": "Maria Santos",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:00:00",
  "tableNumber": 3,
  "numberOfPeople": 2,
  "createdAt": "2026-01-13T18:05:12.7891234Z"
}

# 4. Testar detecção de conflitos (mesma mesa e horário)
$ curl -X POST http://localhost:5000/api/reservations \
-H "Content-Type: application/json" \
-d '{
  "customerName": "Conflicting Reservation",
  "reservationDate": "2026-01-16",
  "reservationTime": "20:15:00",
  "tableNumber": 3,
  "numberOfPeople": 4
}'

HTTP/1.1 409 Conflict
{
  "error": "Table 3 is already reserved for this time slot"
}

# 5. Atualizar reserva
$ curl -X PUT http://localhost:5000/api/reservations/2 \
-H "Content-Type: application/json" \
-d '{
  "customerName": "Maria Santos Silva",
  "reservationDate": "2026-01-16",
  "reservationTime": "21:00:00",
  "tableNumber": 3,
  "numberOfPeople": 3
}' | jq
```

```
{
  "id": 2,
  "customerName": "Maria Santos Silva",
  "reservationDate": "2026-01-16",
  "reservationTime": "21:00:00",
  "tableNumber": 3,
  "numberOfPeople": 3,
  "createdAt": "2026-01-13T18:05:12.7891234Z"
}

# 6. Eliminar reserva
$ curl -X DELETE http://localhost:5000/api/reservations/2

HTTP/1.1 204 No Content
```

Conclusão

Este documento apresentou de forma abrangente a implementação do projeto **DOSFinal - Restaurant Reservation API**, cobrindo todos os aspectos solicitados:

- ❑ **Integração CI/CD:** Pipeline Jenkins completo com 9 stages automatizados
 - ❑ **Containerização:** Docker multi-stage build otimizado
- ❑ **Qualidade de Código:** Integração com SonarQube e Quality Gates
 - ❑ **API RESTful:** 7 endpoints documentados com Swagger
 - ❑ **Testes:** 22 testes unitários com 84.7% de cobertura
- ❑ **Deployment:** Kubernetes com Helm Charts e alta disponibilidade
- ❑ **Documentação:** Detalhes técnicos completos e exemplos práticos

O projeto demonstra a aplicação de **best practices** em desenvolvimento de software moderno, incluindo automação completa do ciclo de vida (CI/CD), qualidade de código, testes robustos e deployment em infraestrutura cloud-native.

DOSFinal Team - 2026

Desenvolvendo soluções robustas e escaláveis com .NET e DevOps